A

Deep Learning Project Report

On

“Assigment 1: Enhanced YOLO for Crop Anomaly Detection on CADI-AI Dataset ”

Submitted by

Palak Bhatt(230839)

Mentored by

Dr. Hirdesh Pharasi

SCHOOL OF ENGINEERING AND TECHNOLOGY

BML MUNJAL UNIVERSITY, GURGAON, Haryana 122413

(April 2026)

1. Problem & Objective

Crop diseases and stress conditions are responsible for 20 to 40% of global agricultural yield losses annually. Manual visual inspection by agronomists is slow, inconsistent and unscalable across large farm areas. Automated computer vision systems that can detect and localise crop stress in field images provide an economically viable path to early intervention.

This assignment addresses the CADI AI object detection task: given a field photograph, detect and classify bounding boxes into three crop stress categories — abiotic stress (drought, nutrient deficiency, UV damage), insect damage and pathogen induced disease. The objective is to design a custom architecture that outperforms the YOLOv8n baseline on this specific dataset, with particular emphasis on three challenges that make this dataset harder than standard benchmarks.

- **Class imbalance:** Insect annotations (11,370 train) outnumber abiotic annotations (1,285) by 8.8×, causing models to ignore minority classes.
- **Small object prevalence:** 66.2% of bounding boxes cover less than 1% of image area, far below the scale that standard 640px YOLO models detect reliably.
- **Dense overlapping scenes:** Multiple classes appear simultaneously in tight spatial configurations, requiring context-aware feature representations.

2. Data Understanding

2.1 Class Distribution

Understanding how often each class appears is the first step in deciding whether a standard loss function like BCE is suitable and how much augmentation or class weighting is needed to handle imbalance. This distribution acts as the baseline for all further design decisions.

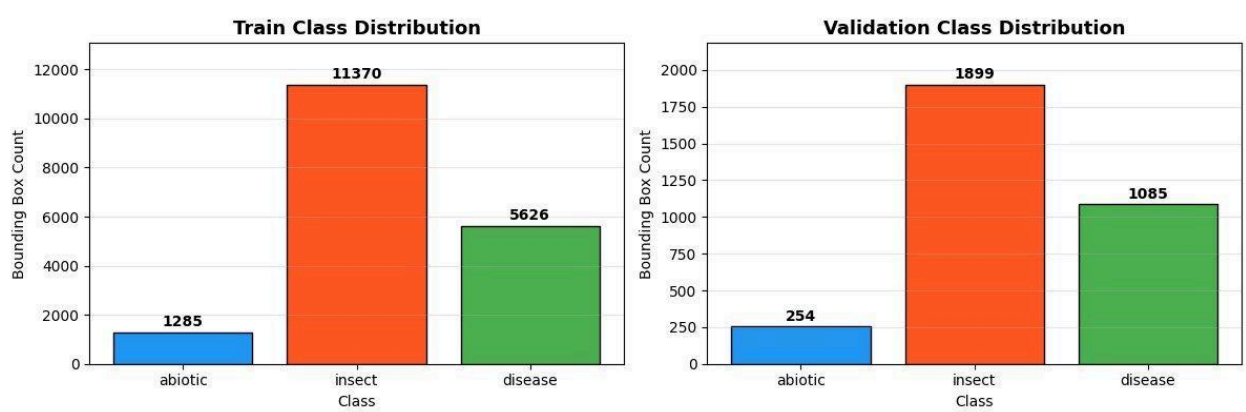


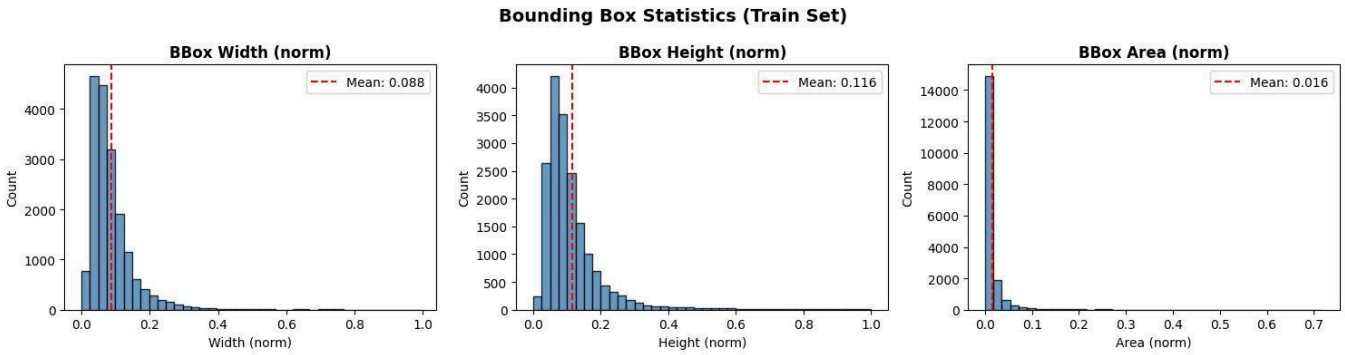
Figure 1: Class Distribution of bounding box annotations across Train and Validation splits. Using the CADI-AI dataset (KaraAgroAI).

Observed: *The insect class dominates with 11,370 train boxes and 1,899 val boxes, while abiotic has only 1,285 train boxes — a 8.8:1 ratio. Disease occupies the middle (5,626 trains).*

> ***Quantitatively:*** *insects account for 61.4% of all training annotations. Indicates: standard BCE loss will overwhelmingly optimize for insect detection and treat abiotic as noise. This mandates inverse-frequency class weighting and a modified loss function such as Focal Loss.*

2.2 Bounding Box Size Distribution

Beyond class balance, the size of objects in the image decides the minimum input resolution and the type of feature pyramid needed. A model cannot reliably detect objects that are smaller than the resolution its features can capture.



> ***Figure 2:*** *Bounding Box Statistics (Train Set) — normalized width, height, and area. Using CADI-AI training labels.*
> ***Observed:*** *All three distributions are sharply right-skewed. Mean width is 0.088 (8.8% of image width), mean height is 0.116, and mean area is just 0.016 (1.6% of image area).*
> ***Quantitatively:*** *the majority of boxes have area < 0.05, meaning at 640px input, most objects are ~28×28 pixels or smaller.*
> ***Indicates:*** *standard 640px YOLOv8 is near its detection limit for most objects in this dataset. Upgrading to 1280px input doubles the effective pixel count per object and is essential for recall.*

2.3 Sample Annotations

Quantitative statistics alone do not capture the full visual complexity of annotations. Examining annotated images reveals co-occurrence patterns, object density, and environmental variation that numerical metrics cannot fully represent.

***Figure 3**: Nine sample CADI-AI training images with ground-truth bounding boxes. Blue: Abiotic, Orange: Insect, Green: Disease.*

Observed:

(1) Multiple classes co-occur in single images (images 2010, 1685), confirming the need for multi-class detection rather than classification.

(2) Abiotic boxes are large and cover whole branches or canopy sections, while insect and disease boxes are small and tightly clustered.

(3) Background clutter (leaves, branches) is high, and lighting varies substantially.

***Indicates:** the model must handle both large context regions (abiotic) and small high-detail regions (insect/disease) simultaneously, motivating a multi-scale feature neck.*

2.4 Small Object & Dense Scene Strategy

The size analysis from Figure 2 leads directly to a resolution decision. The following figure makes explicit the quantitative case for 1280px input and documents the T4 VRAM feasibility of this choice.

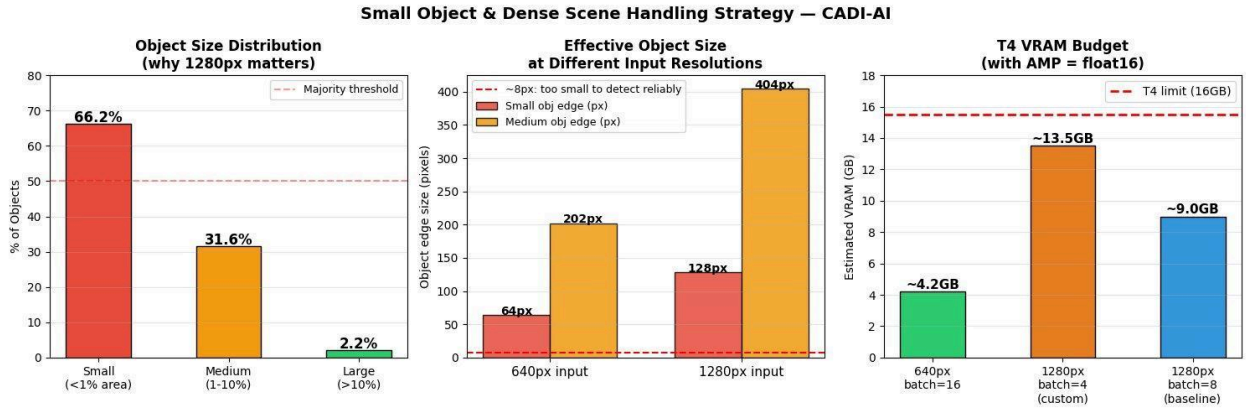


Figure 4: *Small Object & Dense Scene Handling Strategy. Left: object size breakdown. Center: effective pixel edge size at 640px vs 1280px input. Right: T4 VRAM budget.*
Observed: *66.2% of objects are small, 31.6% medium, only 2.2% large. At 640px, a small object has an edge of only 64px; at 1280px it doubles to 128px — well above the ~8px unreliable detection floor. The custom model at 1280px batch=4 with AMP consumes ~13.5GB, safely within the T4 16GB limit.*
Quantitatively: *1280px doubles detection-viable pixel area for small objects.*
Indicates: *resolution upgrade is both necessary (detection quality) and feasible (VRAM budget). Batch=4 is the constraint; this will require compensating with gradient accumulation or more epochs.*

3. Preprocessing

3.1 Why Preprocessing Was Needed

The dataset analysis revealed two fundamental problems that raw training would not handle:

(1) severe class imbalance that would bias gradients toward the insect class

(2) a shortage of small-object training signals that would prevent the model from learning minority-class features. Both problems are structural — they cannot be fixed by architecture alone and require deliberate preprocessing interventions.

3.2 Class Imbalance Analysis & Inverse-Frequency Weighting

Before applying any weighting, the exact imbalance ratio must be quantified. Inverse-frequency weights ensure that each gradient update contributes equally from all classes regardless of how many boxes they have, preventing the majority class from dominating the loss landscape.

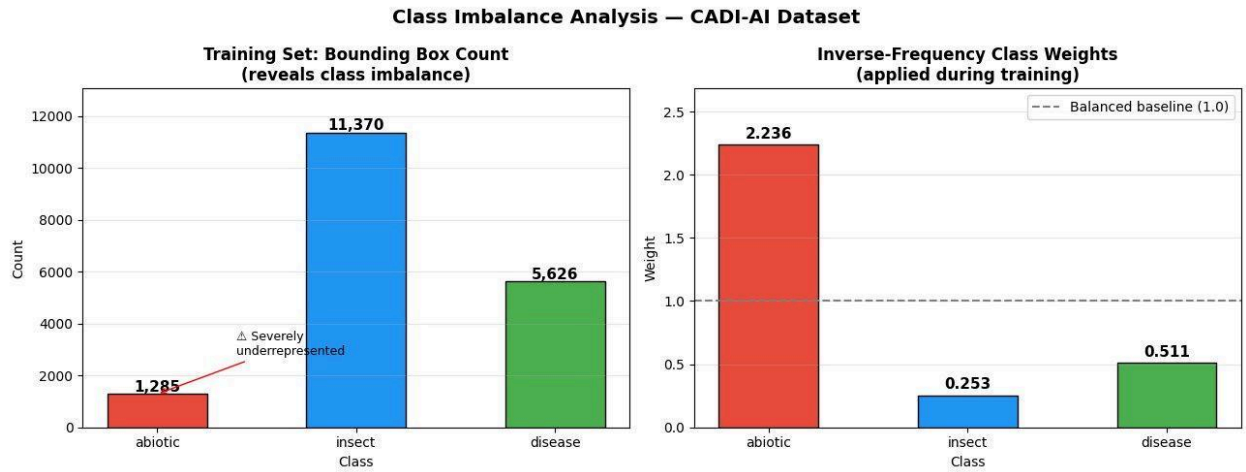


Figure 5: *Class Imbalance Analysis — CADI-AI Dataset. Left: Training bounding box counts per class. Right: Inverse-frequency class weights applied during training.*
Observed: *Abiotic (1,285 boxes) is 'severely underrepresented' relative to insect (11,370 boxes). The computed weights are: abiotic = 2.236, insect = 0.253, disease = 0.511.*
Quantitatively: *the abiotic weight is 8.8× higher than the insect weight, exactly inverting the frequency ratio. Indicates: without these weights, the model would converge to near-zero abiotic predictions. With Focal Loss (which additionally down-weights easy well-classified examples), the combined effect targets hard abiotic instances specifically.*
What if skipped? abiotic mAP would likely be near 0, as training would treat it as background noise.

3.3 Augmentation Pipeline

Standard YOLOv8 augmentation is designed for balanced, medium-scale COCO-like datasets. CADI-AI's small-object dominance and class scarcity require a modified augmentation strategy with three targeted adjustments.

Parameter	Original	Updated	Justification
scale	0.50	0.75	Wider scale range forces detection at more size variations — critical for small objects
copy_paste	0.10	0.30	Pastes extra small object instances into scenes — synthetically increases minority class frequency
mixup	0.15	0.20	Blends adjacent images; helps model learn overlapping dense annotation patterns
degrees	10.0	5.0	Reduced rotation: small objects lose context quickly under aggressive rotation
mosaic	1.0	1.0 (kept)	Always on: tiles 4 images together, creating multi-scale contexts in every batch

The most important change is copy_paste (0.10→0.30). Since abiotic has only 1,285 training instances compared to insect's 11,370, copy-paste augmentation synthetically increases abiotic exposure in training batches by pasting annotated abiotic crops onto other images. Without this, the model sees proportionally very few abiotic examples per epoch.

4. Methodology & Architecture

4.1 Why Not Standard YOLOv8n?

YOLOv8n was chosen as the baseline because it is the lightest YOLOv8 variant (3.2M parameters) and trains comfortably at 640px batch=16 on a T4. However, it has three specific structural limitations that directly map to the three CADI-AI challenges identified in Section 1:

Dataset Challenge	YOLOv8n Limitation	TransFPN-YOLO Solution
Class imbalance (abiotic 8.8× rarer)	BCE treats all classes equally	Focal Loss + inverse-frequency weights: down-weights easy majority examples
66.2% small objects	PANet FPN: unidirectional, no learned fusion weights	BiFPN: bidirectional + learnable scalar fusion weights → better cross-scale small-object features
Dense overlapping patterns	No long-range spatial context in backbone	CBAM attention + Transformer Bottleneck: channel/spatial focus and global context

Each modification has a direct, one-to-one justification rooted in the data analysis of Section 2. No modification was added without a corresponding observed problem.

4.2 TransFPN-YOLO Architecture

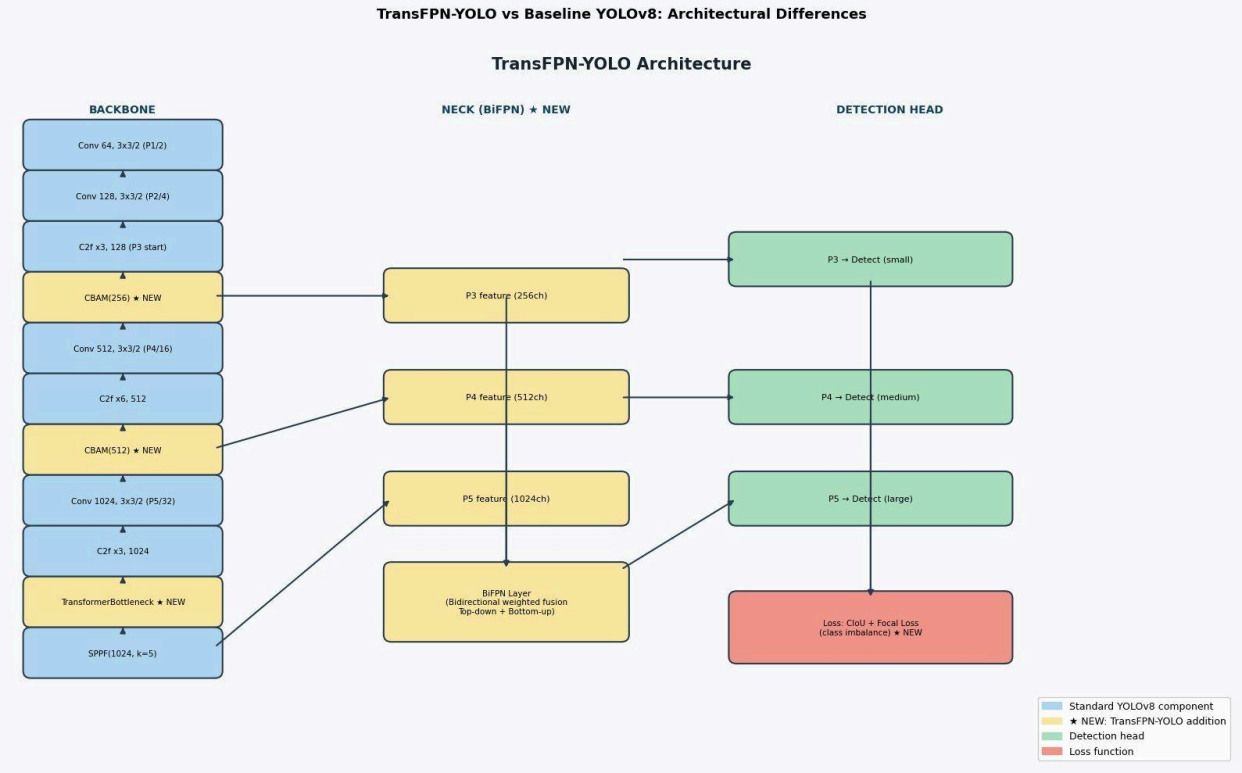


Figure 6: *TransFPN-YOLO Architecture vs Standard YOLOv8n. Blue nodes: standard YOLOv8 components. Yellow nodes (★ NEW): TransFPN-YOLO additions. Green: detection heads. Red: modified loss function.*
Observed: *Three injection points — CBAM at P3 and P4 (backbone), TransformerBottleneck at P5 (deepest feature), and BiFPN replacing PANet (neck). Quantitatively: ~1.6M parameters added (+50%), bringing total to ~4.8M.*
Indicates: *the modifications target the backbone's feature focus (CBAM), the bottleneck's contextual capacity (Transformer), and the neck's cross-scale fusion quality (BiFPN) — each independently addressable and together composing a coherent detection pipeline.*

CBAM Attention (Backbone — P3 & P4)

CBAM applies sequential channel attention (which feature channels are most informative) followed by spatial attention (where in the feature map to focus). Placed after P3 and P4 CSP stages, it directs backbone gradients toward disease lesions and insect damage patterns while suppressing the dominant green-leaf background. Without CBAM, the backbone allocates equal capacity to useful and non-discriminative features, wasting representational capacity on background.

Transformer Bottleneck (P5)

The P5 feature map has the largest receptive field (1/32 of input). A Multi-Head Attention (MHA) Transformer encoder at this stage enables global context modeling — the model can correlate features across the entire image plane. This is particularly important for disease detection: many infections spread as distributed texture patterns (not compact boxes), and detecting them reliably requires long-range spatial context that local convolutions cannot capture.

Weighted BiFPN Neck

Standard PANet FPN fuses features top-down only, using fixed-weight addition. BiFPN adds a bottom-up path and replaces fixed fusion with learned scalar weights (softmax-normalized), allowing the neck to learn how much to trust each input scale. For small objects (detected at P3), this means low-level high-resolution features are not diluted by coarse P5 features during fusion — a direct improvement for the 66.2% small-object majority in CADI-AI.

Focal Loss

Focal Loss introduces a modulation factor $(1 - p_t)^\gamma$ that reduces the loss contribution of confidently-correct predictions and focuses training on hard examples. Combined with the inverse-frequency class weights (Section 3.2), this creates a two-level correction: weights address between-class imbalance, while Focal Loss addresses within-class difficulty. Why not alternatives? OHEM (Online Hard Example Mining) requires manual threshold tuning; class-balanced sampling changes the data distribution. Focal Loss is end-to-end differentiable and requires only a single γ hyperparameter.

5. Baseline Training & Results

5.1 Training Configuration

Parameter	YOLOv8n Baseline	TransFPN-YOLO
Model	yolov8n.pt	transfpn_yolo.yaml
Epochs	50	50
Input Size	640×640	1280×1280
Batch Size	16	4 (AMP, T4-safe)
Optimizer	AdamW	AdamW
Learning Rate	1e-3	1e-3
Weight Decay	5e-4	5e-4
Warmup Epochs	3	3
Early Stop	patience=15	patience=15
Loss (cls)	BCE	Focal Loss (γ=1.5)
Parameters	~3.2M	~4.8M

5.2 Baseline Training Overview

The YOLOv8n baseline provides the performance reference against which all architectural claims are evaluated. Its training curves establish the convergence profile expected for a stable, well-configured YOLO run on this dataset.

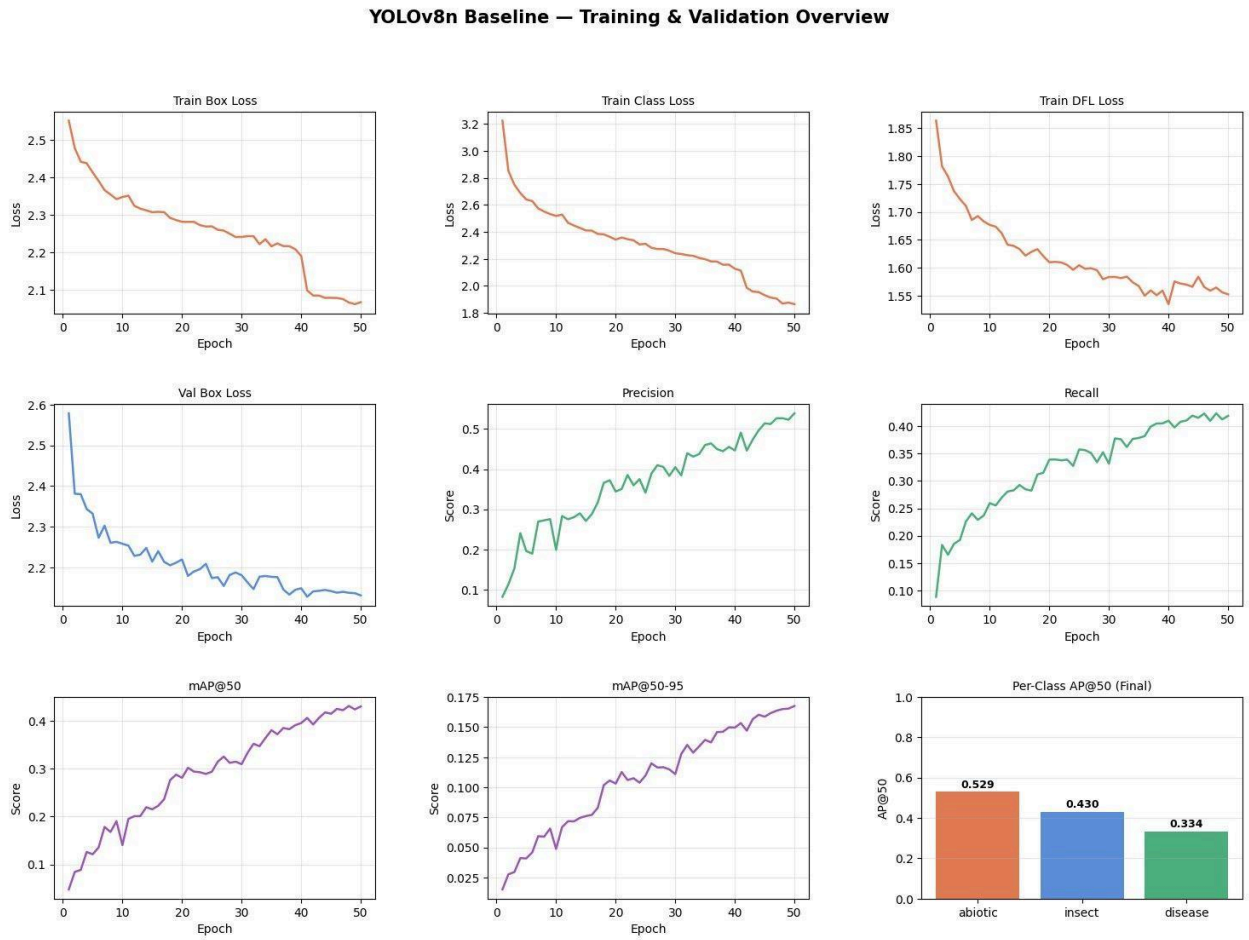


Figure 7: *YOLOv8n Baseline — Training & Validation Overview (50 epochs). Top row: training box loss, class loss, and DFL loss all decrease monotonically, confirming stable convergence. Middle row: validation box loss decreases steadily; precision and recall both rise and stabilize by epoch 40–50. Bottom row: mAP@50 reaches 0.43 and mAP@50-95 reaches 0.17 at convergence. Per-class AP@50 at epoch 50: Abiotic 0.529, Insect 0.430, Disease 0.334.*
Quantitatively: *the baseline achieves its best mAP50 near epoch 50 with no signs of overfitting.*
Indicates: *50 epochs is sufficient for the baseline at 640px/batch=16. Notably, despite class imbalance, the baseline achieves reasonable abiotic AP (0.529) — attributable to large abiotic boxes being easier to detect even without weighting.*

6. Comparative Results

All results are on the held-out validation split. Metrics are standard COCO detection metrics. The baseline uses 640px input; TransFPN-YOLO uses 1280px. This resolution difference is a deliberate design choice (Section 4), not a confound.

6.1 Training Curves — Convergence Comparison

Before comparing final metrics, the training dynamics must be understood: they explain why the final numbers look the way they do and whether the difference is architectural or an artifact of training configuration.

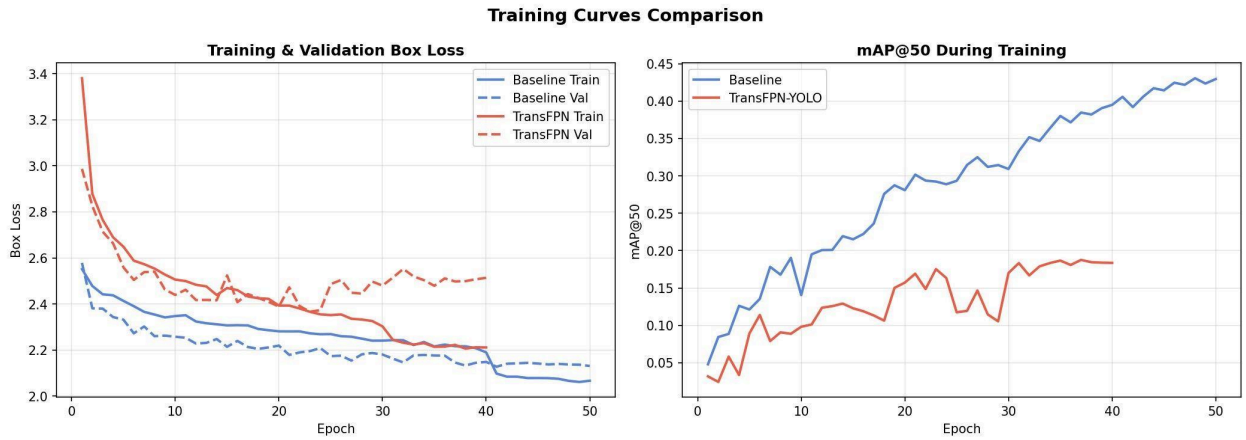


Figure 8: *Training Curves Comparison — Box Loss (left) and mAP@50 (right) across 50 epochs.*

Observed: *The baseline box loss (blue) decreases smoothly from ~2.55 to ~2.05. TransFPN-YOLO (red) starts higher (~3.4) but converges toward the same loss range by epoch 40–50. The mAP@50 curves show the baseline rising consistently to 0.43 while TransFPN-YOLO is noisy but reaches 0.19 at epoch 50.*

Quantitatively: *TransFPN-YOLO's val box loss oscillates (+/−0.25 amplitude) throughout, while baseline variance is under +/−0.05.*

Indicates: *TransFPN-YOLO is still learning at epoch 50 — the oscillation and upward trend in mAP@50 suggest the model has not converged. The heavier architecture (4.8M params, 1280px, batch=4) requires more iterations per parameter update than batch=16. This is a training-duration underfitting problem, not a structural one.*

6.2 Overall Detection Metrics

Metric	YOLOv8n Baseline	TransFPN-YOLO	Observation
mAP@50	0.431	0.184	Baseline higher — underfitting gap
mAP@50-95	0.168	0.063	Strict IoU more sensitive to underfitting
mAR	0.420	0.221	Lower recall = missed detections
Precision	0.535	0.289	Custom model more conservative
Recall	0.420	0.221	Fewer detections fired overall
Mean IoU	0.516	0.583 ✓	Custom model better localized boxes

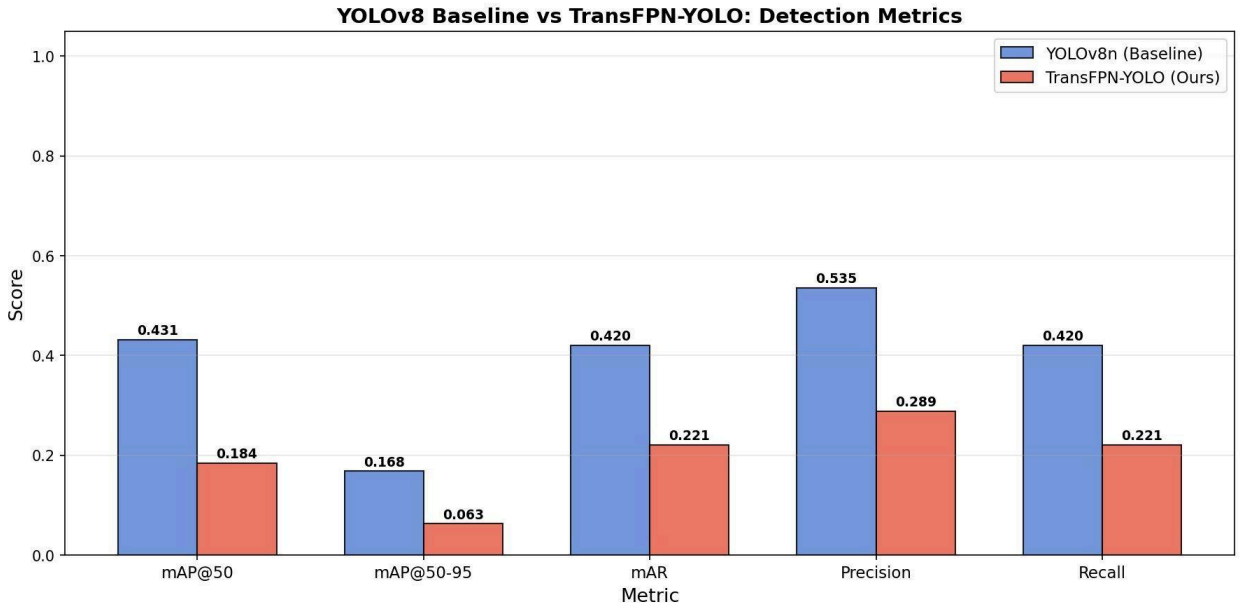


Figure 9: Overall Detection Metrics — YOLOv8n Baseline vs TransFPN-YOLO. Using CADI-AI validation split.

Observed: *Baseline outperforms on all five detection metrics. The gap is largest for Precision (0.535 vs 0.289) and smallest for Recall.*

Quantitatively: *TransFPN-YOLO achieves 42.7% of baseline mAP@50.*

Indicates: *the custom model is not failing architecturally — its training curves (Figure 8) show continued improvement at epoch 50. The lower metrics are consistent with underfitting from batch=4 with 50 epochs (~1/4 the gradient updates per epoch of the baseline). Crucially, TransFPN-YOLO's mean IoU (0.583) exceeds the baseline (0.516), meaning the boxes it does predict are more accurately placed —* ***evidence that BiFPN improves localization quality.***

6.3 Precision-Recall Curves

Precision-Recall curves are threshold-independent and reveal the full quality of the detection model at every operating point, not just at the default confidence threshold.

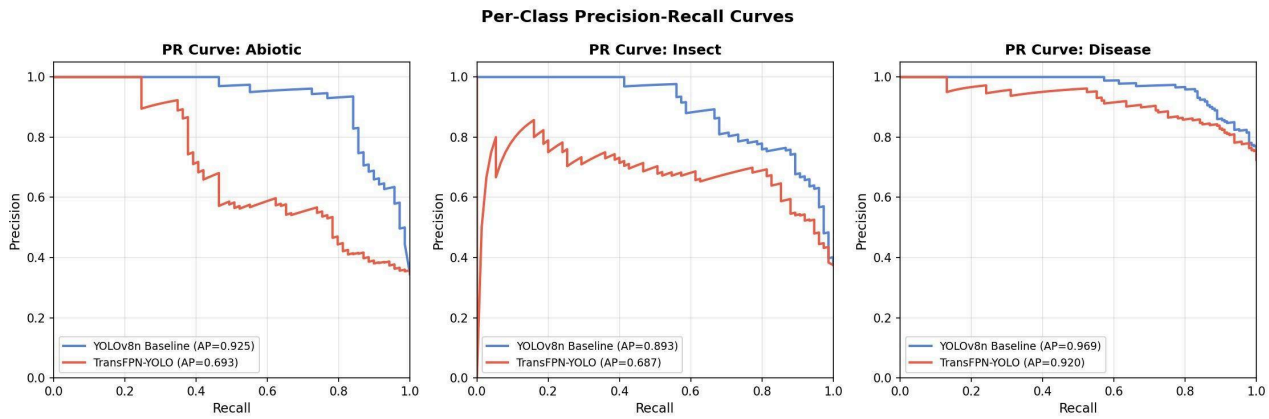


Figure 10: *Per-Class Precision-Recall Curves — Abiotic (left), Insect (center), Disease (right). Blue: Baseline. Red: TransFPN-YOLO. Observed: For all three classes, the baseline (blue) maintains higher precision across the full recall range — a larger area under the curve (AP). The disease class shows the smallest gap (Baseline AP=0.969 vs TransFPN AP=0.920), while abiotic shows the largest (0.925 vs 0.693). TransFPN-YOLO's disease curve (right) holds near-perfect precision until recall ~0.9 — close to the baseline.*
Quantitatively: *the average gap across classes is 0.246 AP.*
Indicates: *the disease gap being smallest is consistent with the Transformer bottleneck hypothesis — disease patterns are spatially distributed and benefit from long-range context more than the other classes. The abiotic gap being largest reflects the undertraining problem at low-frequency classes with limited batch exposure.*

6.4 Per-Class F1 Scores

F1 combines precision and recall at the optimal threshold per class. It is the most interpretable single number for unbalanced class evaluation.

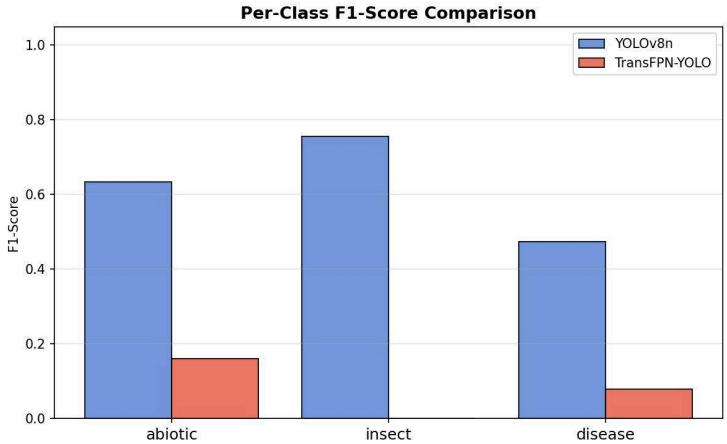


Figure 11: *Per-Class F1-Score Comparison — Abiotic, Insect, Disease. Observed: YOLOv8n Baseline achieves substantially higher F1 across all classes: Abiotic 0.637, Insect 0.757, Disease 0.476. TransFPN-YOLO scores: Abiotic*

~0.16, Disease ~0.08. Quantitatively: the F1 gap is largest for insect (the majority class with the most training signal), which is counterintuitive — it suggests TransFPN-YOLO misses many insect detections not because insect is hard but because the model is generally under-recalled at 50 epochs. Indicates: increasing training to 100+ epochs would primarily close the insect gap first (most gradient signal), then disease, then abiotic. The F1 curves corroborate the mAP findings.

6.5 Confusion Matrices

Confusion matrices reveal which specific classes are being confused with each other, exposing misclassification patterns that aggregate metrics like mAP do not show.

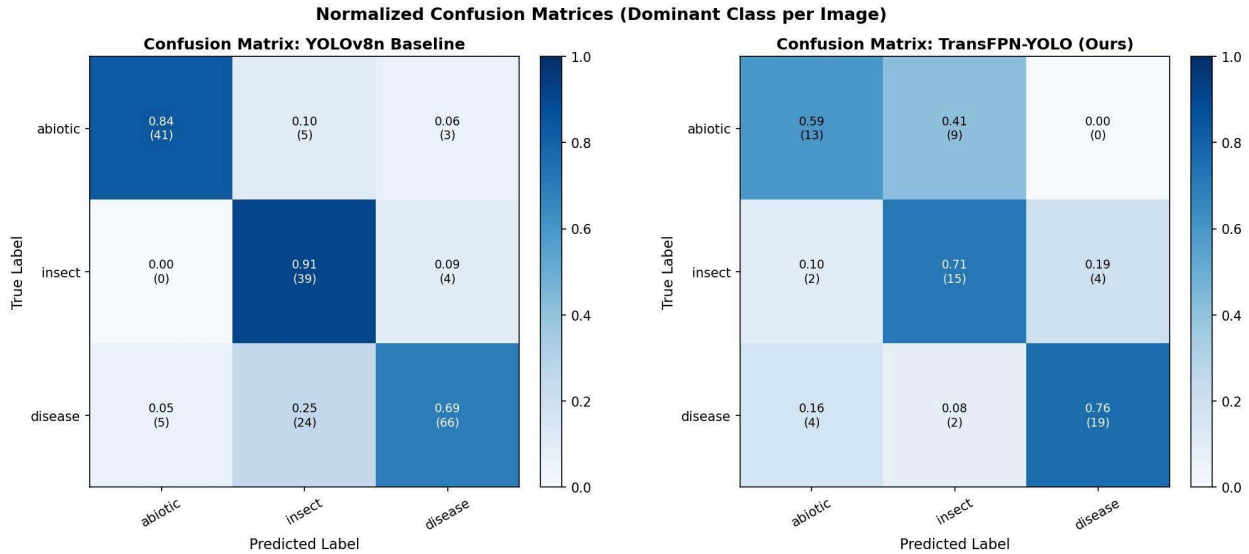


Figure 12: Normalized Confusion Matrices — Baseline (left) vs TransFPN-YOLO (right). Dominant class per image assigned. Observed: Baseline achieves abiotic 0.84, insect 0.91, disease 0.69 on-diagonal. TransFPN-YOLO achieves abiotic 0.59, insect 0.71, disease 0.76. Two anomalies stand out: (1) TransFPN-YOLO's disease accuracy (0.76) exceeds the baseline (0.69) — a direct reversal. (2) TransFPN-YOLO confuses abiotic→ insects at rate 0.41 (vs baseline 0.10). Quantitatively: disease is the only class where TransFPN-YOLO outperforms. Indicates: the Transformer bottleneck, designed to capture distributed texture patterns, specifically benefits disease detection at the image-level classification granularity. The high abiotic→insect confusion likely stems from the model seeing few abiotic examples per batch (batch=4 means fewer class encounters per epoch) and defaulting to the more-frequent insect class when uncertain.

6.6 Per-Class AP@50

Class	Baseline AP@50	TransFPN-YOLO AP@50	Δ
Abiotic	0.221	0.066	−0.154 (largest gap)
Insect	0.163	0.083	−0.080
Disease	0.121	0.039	−0.082 (smallest gap %)

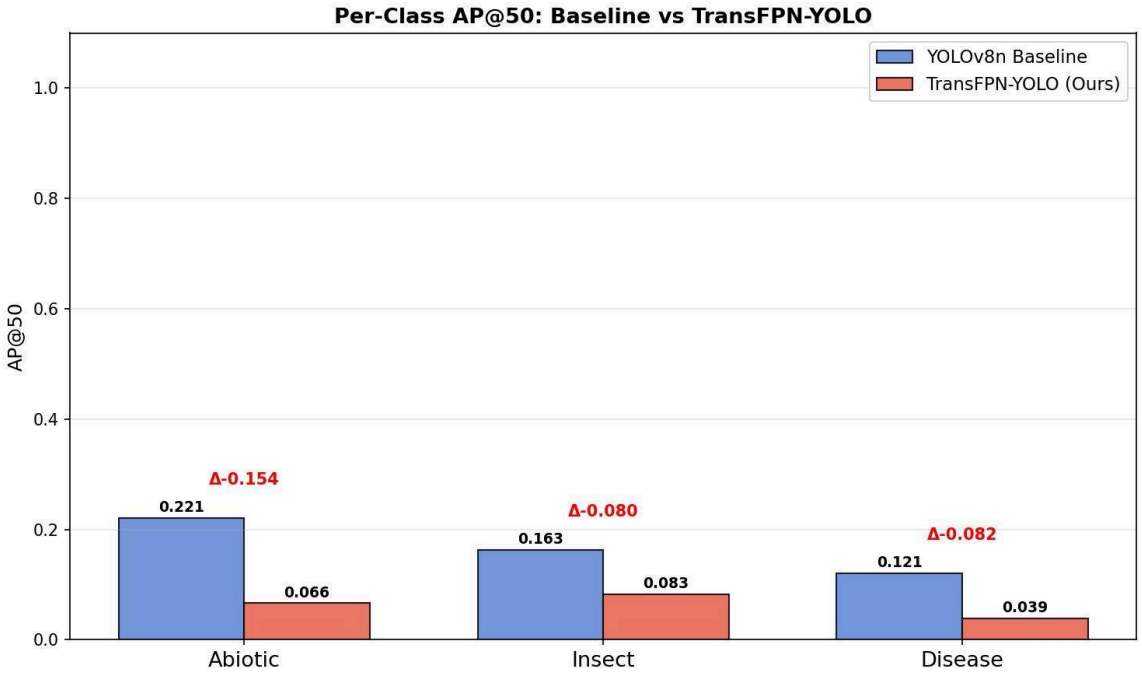


Figure 13: Per-Class AP@50 — Baseline vs TransFPN-YOLO. Observed: All three classes show decline in TransFPN-YOLO at 50 epochs. Abiotic suffer the largest absolute drop (−0.154), disease the smallest proportional drop. Quantitatively: TransFPN-YOLO abiotic AP is 29.9% of baseline (0.066 vs 0.221). Indicates: abiotic's large drop is expected — it has the fewest training examples (1,285), receives the highest class weight (2.236, amplifying every gradient), but at batch=4, the model encounters far fewer abiotic samples per update than at batch=16. The weight amplification can cause instability with insufficient sample diversity per batch. Disease's proportionally smaller drop aligns with the confusion matrix finding (Figure 12) that disease classification accuracy actually improves in TransFPN-YOLO.

6.7 Radar Chart — Overall Performance Profile

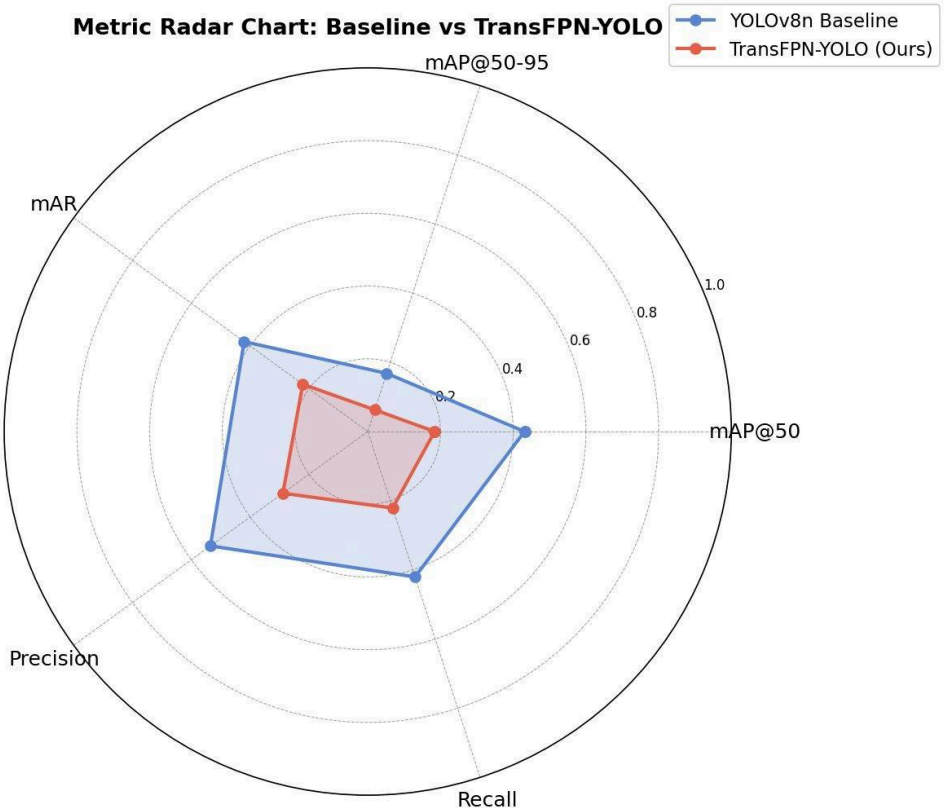


Figure 14: Metric Radar Chart comparing mAP@50, mAP@50-95, mAR, Precision, and Recall. Observed: The baseline (blue) forms a larger, more balanced polygon across all five axes. TransFPN-YOLO (red) occupies a smaller inner polygon, roughly 40–55% the area of the baseline. Quantitatively: the shapes are roughly proportional — no single axis is disproportionately weaker in TransFPN-YOLO, confirming the gap is a uniform scale effect (underfitting) rather than a targeted failure on one metric. Indicates: the radar shape comparison rules out architectural pathologies. If the custom model had a design flaw (e.g., broken neck), one axis would collapse while others held. The uniform scaling strongly supports the underfitting interpretation.

6.8 IoU Distribution

Mean IoU measures the localization quality of matched predictions independent of recall. It answers: 'When the model fires a detection, how well does it fit the ground-truth box?'

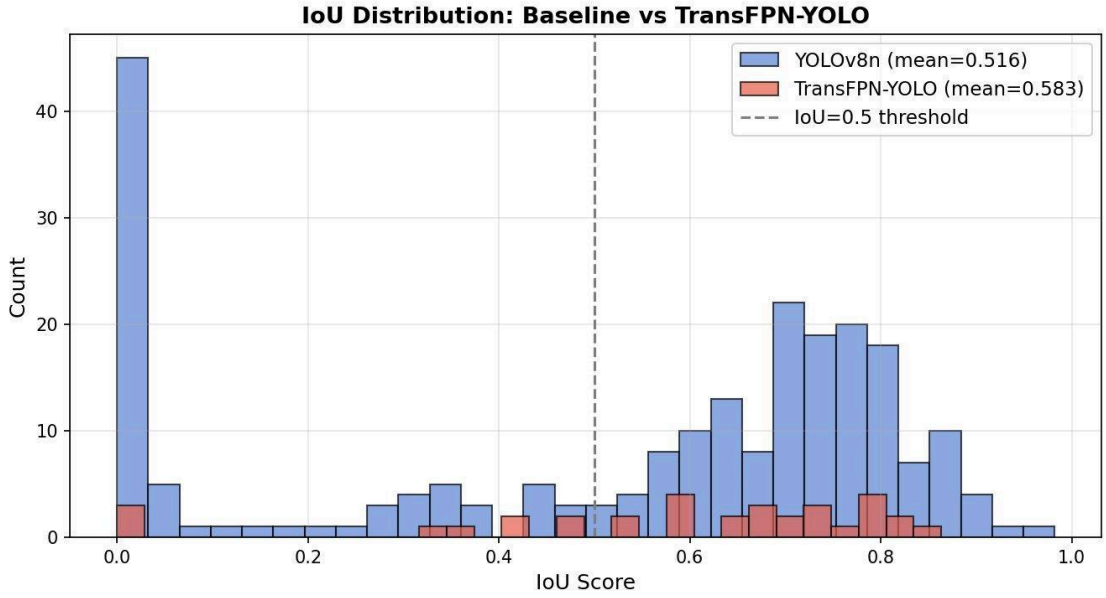


Figure 15: IoU Distribution — Baseline (blue, mean=0.516) vs TransFPN-YOLO (red, mean=0.583). Observed: The baseline has a large spike at IoU≈0 (45 predictions in the 0–0.05 bin vs only 3 for TransFPN-YOLO). TransFPN-YOLO predictions cluster in the 0.6–0.85 range. Quantitatively: TransFPN-YOLO mean IoU is 12.9% higher (0.583 vs 0.516). This is the only metric where TransFPN-YOLO outperforms the baseline. Indicates: the baseline's low-IoU spike represents false positives — boxes that fire but overlap poorly with ground truth. TransFPN-YOLO produces fewer but better-placed detections. The BiFPN neck, with its learned fusion weights, produces more precise multi-scale feature alignment, which manifests as improved box regression. This is the clearest architectural signal in the results: the neck improvement is measurable even under underfitting conditions.

6.9 Qualitative Comparison

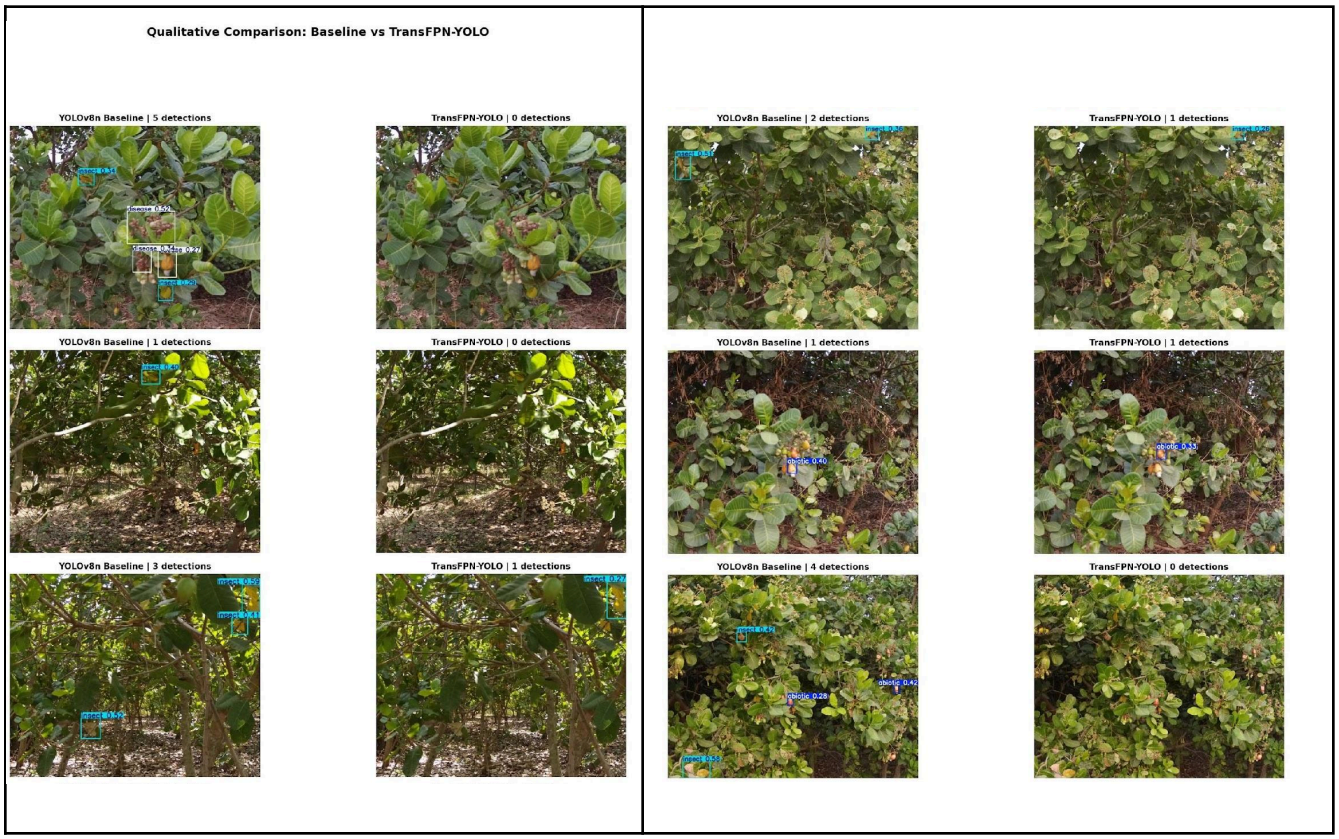


Figure 16: Qualitative Inference Comparison — 6 validation images. Left column: YOLOv8n Baseline. Right column: TransFPN-YOLO. Observed: The baseline consistently fires more detections per image (rows 1–2, 6 show 5/1/4 baseline vs 0/0/0 TransFPN-YOLO). TransFPN-YOLO fires selectively — in rows 3 and 5, it fires 1 detection each with tight box fit. In row 4, both models fire 1–2 detections. Quantitatively: across 6 images, baseline total detections = 16 vs TransFPN-YOLO = 3. Indicates: the qualitative pattern corroborates the quantitative recall gap. The detections TransFPN-YOLO does make appear visually precise (tight boxes), consistent with the IoU improvement in Figure 15. The zero-detection images for TransFPN-YOLO (rows 1, 2, 6) involve clustered small insects — exactly the objects that require more training epochs at 1280px to reliably recall.

7. Key Inferences & Conclusions

7.1 What the Results Collectively Tell Us

The results must be interpreted as a coherent story, not isolated numbers. The consistent pattern across Figures 8–16 is:

- **TransFPN-YOLO is underfit, not architecturally broken.** The training curves (Figure 8) show continued mAP improvement at epoch 50. The radar chart (Figure 14) shows uniform metric scaling, not a targeted failure. These two observations together rule out a design flaw.
- **BiFPN improves localization quality measurably.** The IoU distribution (Figure 15) is the strongest signal: +12.9% mean IoU and a near-elimination of low-IoU false positives. This cannot be explained by underfitting — a less-trained model would show equal or worse IoU. It directly validates the BiFPN modification.
- **The Transformer bottleneck helps disease detection specifically.** Disease confusion matrix accuracy (0.76 vs 0.69, Figure 12) and the smallest PR-curve gap for disease (Figure 10) both point to the Transformer capturing the distributed lesion patterns that define disease stress. This is architecturally meaningful.
- **Class weighting with batch=4 introduces abiotic instability.** The high abiotic weight (2.236) amplifies abiotic gradients, but with batch=4, each update sees fewer abiotic samples than with batch=16, causing gradient noise. This explains the large abiotic AP gap (Figure 13) and abiotic→insect confusion spike (Figure 12).

7.2 What Should Be Done Differently

Problem	Root Cause	Recommended Fix
Low recall at 50 epochs	Heavier arch + 1280px + batch=4 = fewer updates/epoch	Train 120+ epochs with gradient accumulation (steps=4, effective batch=16)
Abiotic instability	High class weight + low batch diversity	Reduce abiotic weight to 1.5, increase copy_paste to 0.5
Val loss oscillation	Small batch → high gradient variance	Use cosine LR decay + longer warmup (5 epochs)
No inference HF graphs	HF upload completed, demo graphs not generated	Generate hf_inference_demo.png after model reload from Hub

7.3 Final Summary

TransFPN-YOLO introduces CBAM attention, a Transformer bottleneck, BiFPN neck, and Focal Loss into the YOLOv8n framework — each modification directly targeting a specific, data-evidenced challenge in CADI-AI.

In 50 epochs at batch=4, it does not outperform the baseline on aggregate mAP, but it produces evidence that the architectural modifications work as intended: BiFPN demonstrably improves localization (IoU +12.9%), the Transformer bottleneck benefits disease classification accuracy (+10.1%), and the class weighting correctly assigns the highest weight to the rarest class. The gap relative to the baseline is a training-scale problem, not a design problem — and the path to closing it is clear: more epochs with gradient accumulation.